The Cost of Being Wrong

A Cost-Sensitive Fraud Risk Model from the Mobile Transactions Dataset

Technical Report

Prepared by:	Diana Adhiambo Owuoth - 093481
Course:	MSc Data Science & Analytics - Applied Machine Learning
Assignment:	DSA 8401, Assignment 2 - The Cost of Being Wrong
Date:	September 5, 2026

Introduction

This report builds a model that looks at a mobile money transaction and decides how likely it is to be fraudulent, and just as important, decides at what point that suspicion is strong enough to act on. Every fraud detection system makes two kinds of mistakes, letting a real fraud case slip through, or wrongly flagging a genuine transaction. Neither is free. Here, a missed fraud case costs KES 10,000, since that money is usually gone for good, while a wrongly flagged transaction costs KES 800, mostly the inconvenience to a genuine customer. That 12.5 times difference in cost is the thread running through this whole report, it means the model with the highest accuracy is not automatically the model that saves the most money, which Section 1 shows directly.

1 Evaluation plan

Assignment brief: set up fair testing for imbalanced, time-ordered data and justify the chosen metric.

Our data runs from February 2025 to July 2026, 180,000 transactions once duplicates are removed, all time stamped. We split it into five folds in time order, each trained only on data from before its test window, with a 30 day gap between the two, since several of our features are 30 day rolling counts and a gap stops information from the test period leaking backward. This mirrors real use, the model only ever sees the past.

For scoring, we used PR-AUC over ROC-AUC. A model guessing at random would score about 0.5 on ROC-AUC and about 0.084 on PR-AUC, since 0.084 is simply our actual fraud rate. ROC-AUC gives credit for correctly ignoring the 91.6 percent of transactions that were never going to be flagged anyway, the easy part. PR-AUC only credits actually finding fraud, the hard part, with nowhere to hide behind the majority class. Since finding fraud is the entire point, PR-AUC is our decision metric.

This is also why the best accuracy model is rarely the best production model. A model that always predicts not fraud would be right 91.6 percent of the time, and would miss all 12,572 fraud cases in our evaluation set, costing KES 125,720,000. Our actual chosen model, at the threshold recommended in Section 3, is only right 57.7 percent of the time, but because it catches most of the fraud, it costs KES 101,238,800, over KES 24 million cheaper. Accuracy rewards being right on the easy majority, cost cares about the rare cases that actually hurt.

2 Ensemble model comparison

Assignment brief: compare bagging and boosting ensembles on identical folds, discussing bias versus variance.

We compared a straightforward logistic regression against two ensemble types, bagging in random forest and boosting in XGBoost, on identical folds.

Model	**PR-AUC**	**ROC-AUC**
Logistic regression	0.1178	0.5918
Random forest	0.1058	0.5602
XGBoost, default	0.0983	0.5368
XGBoost, tuned	0.1191	0.5928

With default settings, both tree models underperform plain logistic regression and their scores swing three to four times more across folds. The underlying fraud signal is weak, even the best untuned model barely clears the 0.084 no skill floor, so a high capacity model like an unconstrained XGBoost has room to fit patterns specific to one time period that don't hold up in the next. Logistic regression is too simple to do that, so it stays consistent. Once XGBoost is tuned in Section 4, forcing it shallow and adding regularisation, it pulls ahead of logistic regression with less variance than either tree model, boosting's advantage only shows up once it's stopped from overfitting.

3 Cost based decision threshold

Assignment brief: find the cost-minimising threshold and show savings per 1,000 transactions.

A smoke detector that goes off at every slice of toast is annoying, one that stays silent during a real fire is a disaster, the two mistakes are never equally bad, and the same logic applies here.

At the default 0.5 cutoff, our model produced 7,179 missed frauds and 39,993 false alarms, five times more false alarms than misses. But the missed frauds cost 7,179 times 10,000, KES 71,790,000, against the false alarms' 39,993 times 800, KES 31,994,400. Despite being the smaller group, missed frauds account for close to 70 percent of the total cost, which is why minimising the count of mistakes is not the same as minimising their cost.

Cutoff	**Threshold**	**Missed frauds**	**Flagged wrongly**	**Total cost (KES)**	**Savings /1,000**
Default 0.50	0.50	7,179	39,993	103,784,400	—
Cost optimal, raw	0.47	4,858	65,623	101,078,400	18,022
Cost optimal, calibrated	0.08	5,481	58,036	101,238,800	16,953

We recommend the calibrated threshold of 0.08, Section 5 explains why that's more trustworthy than the raw score version despite saving marginally less. Worth noting, the same search run on an uncalibrated model gave a threshold of 0.02, flagging 91 percent of all transactions, mathematically optimal and operationally useless, which is exactly why calibration matters as much as the threshold search itself.

4 Imbalance handling and tuning

Assignment brief: compare imbalance methods correctly inside each fold, prove leakage inflates scores, and report an honest tuning search.

Teaching a model to spot fraud from data where only 8 in 100 cases are fraud is like teaching someone to spot a rare bird from a thousand photos where the bird appears eight times, without help, the safe guess is always "no bird."

We compared class weighting, which tells the model to treat fraud mistakes as more costly during training, against SMOTE, which generates synthetic fraud examples by blending real ones. Done correctly, SMOTE is fit only on training data. Done incorrectly, fitting it before the train and test split, a synthetic training row can be built from a real case sitting in the test set.

Method	**PR-AUC**	**ROC-AUC**
Class weighting	0.0983	0.5368
SMOTE, correct	0.1023	0.5533
SMOTE, leaked	1.0000	undefined

SMOTE done properly gives a small, honest lift over class weighting. Done incorrectly it gives a perfect score, which is the warning sign itself, no real fraud model is ever perfect, so a perfect result almost always means the test was compromised. We include this to show our reported results are free of it.

For tuning, we used Optuna to automatically search XGBoost's settings within a 60 trial budget, with pruning to cut off clearly weak attempts early. 67 trials completed, since the search checkpoints its progress and an earlier interrupted run resumed rather than restarting. The strongest combination found was deliberately restrained, shallow trees, a slow learning rate, and strong regularisation, confirming that the fix for Section 2's overfitting was restraint, not complexity.

5 Calibration

Assignment brief: check calibration with a reliability diagram and Brier score, and recalculate the threshold after.

A weather forecaster who says 70 percent chance of rain should be right roughly 7 times out of 10 on those days, otherwise their number can't be trusted even if their yes or no calls seem reasonable. This section checks whether our model's confidence scores behave the same way.

Class weighting, needed to handle the imbalance, pushes predicted probabilities away from what they actually mean. We measured this with the Brier score, lower is more honest, zero is perfect.

Model	Brier, raw	Brier, calibrated	Threshold after
Logistic regression	0.2442	0.0760	0.08
Random forest	0.1718	0.0764	0.08
XGBoost, untuned	0.1851	0.0766	0.04
XGBoost, tuned	0.2395	0.0759	0.08

Raw scores of 0.5 corresponded to an actual fraud rate of only 10 to 14 percent. After isotonic calibration, every model's Brier score dropped to around 0.076, and the cost optimal thresholds stopped scattering between 0.02 and 0.47 and settled into a tight 0.04 to 0.08 range. There's an independent check on this too, a properly calibrated classifier's theoretical best threshold is the false alarm cost divided by the sum of both costs, 800 divided by 10,800, about 0.074. Our calibrated threshold of 0.08 lands almost exactly there, arrived at from a completely different direction, strong evidence the calibration genuinely fixed the scores.

6 Explainability and fairness

Assignment brief: produce global and local SHAP explanations and subgroup fairness metrics against regulatory expectations.

A good airport security officer can tell you exactly why they pulled someone aside, and it would be a real problem if one nationality were checked far less accurately than another for no behavioural reason. This section asks our model the same two questions.

Using SHAP, the strongest fraud signal by far was whether a transaction happened at night, between midnight and 5am, followed by a high recent cashout ratio and small transaction size, small late night cash withdrawals is a known pattern in how compromised accounts get emptied out. For one transaction flagged at 81 percent, being a night transaction alone added 0.39 to that score and the cashout ratio added 0.26, accounting for most of the decision.

For fairness we checked region and account age as a tenure proxy. The newest account age bands had too little data in our test window, seven and one transactions, to draw conclusions from. Region was clearer.

Region	Approval rate	False negative rate
Mwanza, best	0.580	0.368
Arusha, worst	0.605	0.514
All others	0.586 – 0.602	0.41 – 0.44

Approval rates stayed close across regions, 0.58 to 0.60, but false negative rates ranged from 37 percent in Mwanza to over half in Arusha, meaning a genuine fraud case in Arusha is meaningfully more likely to succeed, worth investigating before deployment.

The assignment references the CBK Digital Credit Providers Regulations of 2022, brought in after unregulated lending apps caused real harm in Kenya. In plain terms it requires lenders to disclose loan terms upfront, give notice before changes, share transaction history on request, never harass over debt, and provide a real complaints channel, all pointed at one idea, a customer should never be blindsided by a financial decision. We're not a lender so this doesn't apply directly, but the same principle carries through CBK's fraud and cybersecurity guidance and the Data Protection Act 2019, both of which expect a customer to get a real answer when a transaction is blocked. Our SHAP output gives that answer per transaction, the Arusha gap is where the same principle applies at a group level, and we don't yet have that answer.

7 Recommendation

Assignment brief: recommend a single model and threshold for deployment.

We recommend deploying the tuned XGBoost model at the calibrated threshold of 0.08.

Why this model. It had the best PR-AUC of everything tested, 0.1191, and the least variation across folds, a model that scores well once but swings wildly on the next batch isn't one worth trusting.

Why this threshold. The default 0.5 cutoff costs more than necessary, and the calibrated threshold landed almost exactly on the theoretical cost optimal value of 0.074, two independent routes arriving at the same answer.

Why not the alternatives. The untuned model's own cost optimal threshold flagged 91 percent of all transactions, not a usable system. The raw score threshold saved marginally more on paper, but only using confidence scores already shown to be unreliable.

The gap we're not hiding. This model misses 51.4 percent of real fraud in Arusha against 36.8 percent in Mwanza, meaningfully worse protection for that region, and we don't yet know if that's a genuine pattern difference or a model weakness.

Before this goes live, two things should happen first, an investigation into the Arusha gap, since the fix depends on the cause, and a hard cap on review queue volume, since the cost formula alone could push toward unrealistic flag rates if fraud patterns shift after deployment. The goal was never the single highest score, it was the model that makes the fewest expensive mistakes, can explain itself, and is honest about where it still falls short.